

第X章：深入理解字面量 (Literals)

1. 定义与核心概念 (Definition)

- **什么是字面量**：源代码中固定值的直接表示法（如 `12`, `0x5A`, `"Error"`）。
- **右值特性**：字面量通常是“右值 (R-value)”，没有内存地址（字符串除外），不可被赋值。
- **与 `const` 变量的区别**：
 - 字面量：不仅只读，而且通常没有独立的存储实体（寄生在指令中）。
 - `const` 变量：本质是只读变量，有具体的内存地址。

2. 书写规范与类型推导 (Syntax & Suffixes)

重点：编译器如何“看”一个数字，决定了它在运算时的行为。

- **进制前缀 (Radix)**
 - 十六进制 (`0x` / `0X`)：寄存器配置首选。
 - 二进制 (`0b` / `0B`)：C23 标准或 GCC 扩展，位掩码配置神器。
 - 八进制 (`0`)：**高危陷阱**（如 `010` 等于 8），需特别警惕。
- **关键后缀 (Suffixes)**
 - `U` / `u` (**Unsigned**)：**嵌入式必用**。防止移位操作时的符号位扩展错误 (Undefined Behavior)。
 - `L` / `LL` (**Long**)：防止大数运算时的溢出截断。
 - `F` / `f` (**Float**)：强制单精度。不加默认为 `double`，可能导致 MCU 进行软浮点模拟，严重拖慢中断响应。

3. 字面量的底层存储机制 (Underlying Storage)

核心逻辑：代码里写的数字，到底存在 Flash 还是 RAM？

3.1 小整数：指令内嵌 (Immediate Value)

- **机制**：作为立即数直接编码在机器指令 (Opcode) 中。
- **存储位置**：程序存储器 (Flash/ROM) 的指令段 (Text Segment)。
- **特点**：速度极快，不占用任何数据 RAM。
- **汇编示例**：`MOV R0, #0x12`

3.2 大整数与浮点数：文字池 (Literal Pool)

- **机制**：当数字超过指令位宽限制（如 32位大数无法塞入 32位指令）时，编译器将其存放在代码段末尾。
- **存储位置**：程序存储器 (Flash) 的文字池区域。
- **读取方式**：PC 相对寻址 (`LDR R0, [PC, #offset]`)。
- **特点**：依然不占 RAM，但比立即数多一次访存操作。

4. 特殊的字面量：字符串 (String Literals)

- **本质**：匿名数组，存储的是字符序列。
- **存储位置**：
 - 通常位于只读数据段 (`.rodata`)。

- 在嵌入式系统中映射在 Flash 中。
- **内存占用**：这是**唯一**占用独立线性地址空间的字面量类型。
- **退化行为**：在表达式中自动退化为指向首字符的指针 (`char*`)。

5. 进阶：复合字面量 (Compound Literals - C99)

- **语法**：`(type){ list }`，例如 `(struct Config){.enable=1, .mode=2}`。
- **应用场景**：
 - 驱动初始化传参：`hal_init(&(uart_cfg_t){9600, 8, 'N', 1});`
 - 避免定义临时变量，代码更紧凑。
- **作用域与存储**：
 - 函数内使用：通常在栈 (Stack) 上临时创建。
 - 全局/静态使用：在数据段 (.data/.bss) 创建。